

Métaheuristiques pour le Set Packing Problem

Exercices d'Implémentation

NAIT SLIMANI Maroua & MOLLI Lila & ABID Ikram

Master Informatique

Nantes Université

Année 2025-2026

Table des matières

1	EI1 : Heuristiques Constructives	3
1.1	Heuristique gloutonne	3
1.2	Recherche locale par descente	3
1.3	Instances testées	3
1.4	Résultats numériques	3
1.5	Analyse graphique	5
1.5.1	Graphique 1 : Évolution des solutions	5
1.5.2	Graphique 2 : Amélioration apportée par la recherche locale	6
1.5.3	Graphique 3 : Comparaison des temps de calcul	7
1.5.4	Graphique 4 : Gap d'optimalité	8
1.6	Conclusion EI1	8
2	EI2 : GRASP et Reactive GRASP	8
2.1	Importance du paramétrage	8
2.1.1	Choix de α pour GRASP Standard	8
2.1.2	Avantages de Reactive GRASP	9
2.2	Protocole expérimental	9
2.3	Résultats numériques	9
2.4	Analyse graphique	10
2.4.1	Graphique 1 : Comparaison Z moyen	10
2.4.2	Graphique 2 : Variabilité des solutions	11
2.4.3	Graphique 3 : Comparaison temps moyen	12
2.4.4	Graphique 4 : Gain de Reactive GRASP	13
2.5	Conclusion EI2	13
3	EI3 : Battle des MétaHeuristiques ACO vs GRASP vs GA	13
3.1	Contexte et objectif	13
3.2	Métaheuristiques comparées	13
3.2.1	GRASP (Greedy Randomized Adaptive Search Procedure)	13
3.2.2	ACO (Ant Colony Optimization)	14
3.2.3	GA (Genetic Algorithm)	14
3.3	Protocole expérimental	14
3.4	Résultats numériques	14
3.5	Analyse comparative	16
3.5.1	Graphique 1 : Comparaison de la qualité moyenne	16
3.5.2	Graphique 2 : Comparaison des temps d'exécution	17

3.5.3	Graphique 3 : Robustesse (écart-type)	18
3.5.4	Graphique 4 : Efficacité (ratio qualité/temps)	19
3.5.5	Graphique 5 : Courbes de convergence	20
3.6	Statistiques globales	21
3.7	Discussion :	21
3.7.1	Domination d'ACO	21
3.7.2	Performance de GRASP	21
3.7.3	Analyse critique de l'AG : Forces et Faiblesses	21
3.7.4	Quand utiliser chaque algorithme ?	22
3.8	Conclusion EI3	22
A	Implémentation	23
A.1	Structure du code	23
A.2	Utilisation	23

1 EI1 : Heuristiques Constructives

1.1 Heuristique gloutonne

L'heuristique gloutonne construit une solution de manière itérative en sélectionnant à chaque étape la variable ayant la meilleure utilité. L'utilité d'une variable j est définie par :

$$u_j = \frac{c_j}{\sum_{i=1}^m a_{ij}} \quad (1)$$

1.2 Recherche locale par descente

La phase d'amélioration utilise une recherche locale par descente profonde avec deux types de voisinages :

- **Voisinage 1-1** : retirer 1 variable sélectionnée, ajouter 1 nouvelle variable
- **Voisinage 2-1** : retirer 2 variables sélectionnées, ajouter 1 nouvelle variable

L'algorithme explore l'ensemble du voisinage à chaque itération et applique le meilleur mouvement trouvé jusqu'à atteindre un optimum local.

1.3 Instances testées

Nous avons évalué nos heuristiques sur 10 instances de tailles variées, triées par complexité croissante (de didactic 7×9 jusqu'à pb_2000rnd0100 10000×2000).

1.4 Résultats numériques

TABLE 1 – Résultats EI1 : Glouton + Recherche Locale

Instance	m	n	Z*	T(s)	Statut	Z	T(s)	Gap%	Z	T(s)	Gap%	Speedup
didactic.dat	7	9	—	—	N/A	30	0.0	—	30	0.0	—	—
pb_100rnd0100.dat	500	100	372	0.64	OPTIMAL	342	0.0023	8.06	348	0.0114	6.45	56.5
pb_100rnd0300.dat	500	100	—	—	N/A	193	0.0111	—	194	0.0131	—	—
pb_200rnd0100.dat	1000	200	—	—	N/A	351	0.0082	—	359	0.0257	—	—
pb_200rnd0300.dat	1000	200	—	—	N/A	682	0.034	—	688	0.1596	—	—
pb_500rnd1500.dat	1500	500	—	—	N/A	1059	0.0974	—	1086	0.2023	—	—
pb_500rnd0100.dat	2500	500	—	—	N/A	285	0.0371	—	285	0.0461	—	—
pb_500rnd0300.dat	2500	500	—	—	N/A	674	0.1921	—	674	0.2271	—	—
pb_1000rnd0100.dat	5000	1000	—	—	N/A	49	0.0812	—	49	0.083	—	—
pb_2000rnd0100.dat	10000	2000	—	—	N/A	37	0.5417	—	37	0.5491	—	—

1.5 Analyse graphique

1.5.1 Graphique 1 : Évolution des solutions

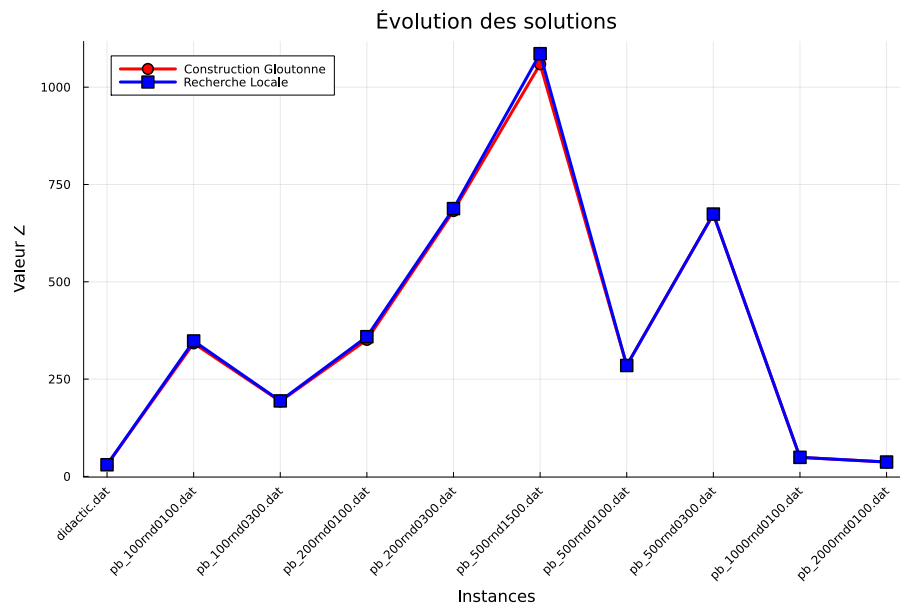


FIGURE 1 – Évolution Construction Gloutonne vs Recherche Locale

Interprétation :

On observe que la recherche locale améliore la solution initiale construite par l'heuristique gloutonne. L'amélioration est particulièrement visible sur l'instance `pb_500rnd1500.dat` où le gain est de 27 points. Pour certaines instances comme `didactic`, les deux courbes se superposent car la construction atteint déjà un optimum local.

1.5.2 Graphique 2 : Amélioration apportée par la recherche locale

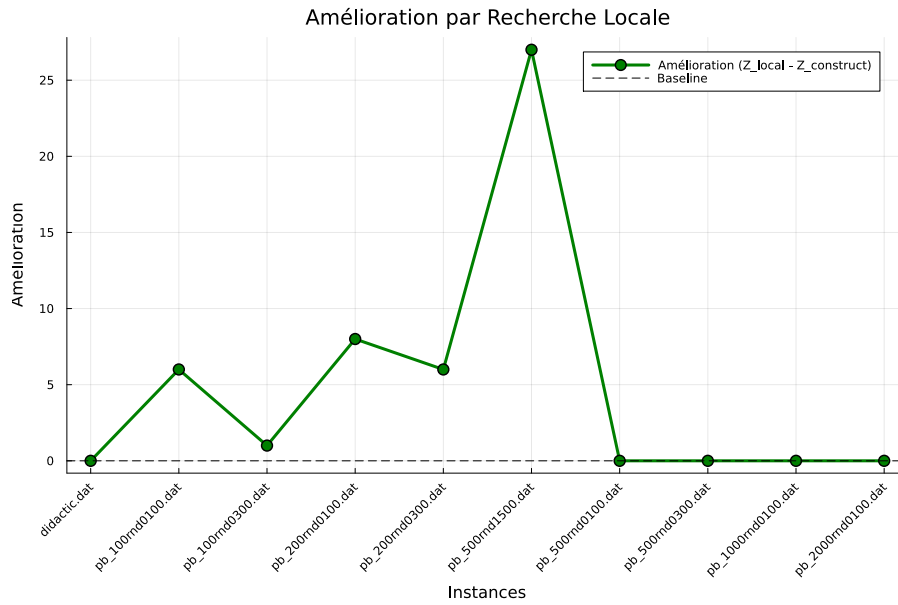


FIGURE 2 – Amélioration (Z_{local} - Z_{construct})

Interprétation :

Ce graphique montre clairement l'apport de la recherche locale. Les améliorations les plus importantes sont observées sur les instances de taille moyenne (pb_200rnd0100, pb_500rnd1500). Pour les très grandes instances (pb_1000rnd0100, pb_2000rnd0100), l'amélioration est nulle ou très faible, suggérant que le glouton atteint rapidement un optimum local difficile à améliorer.

1.5.3 Graphique 3 : Comparaison des temps de calcul

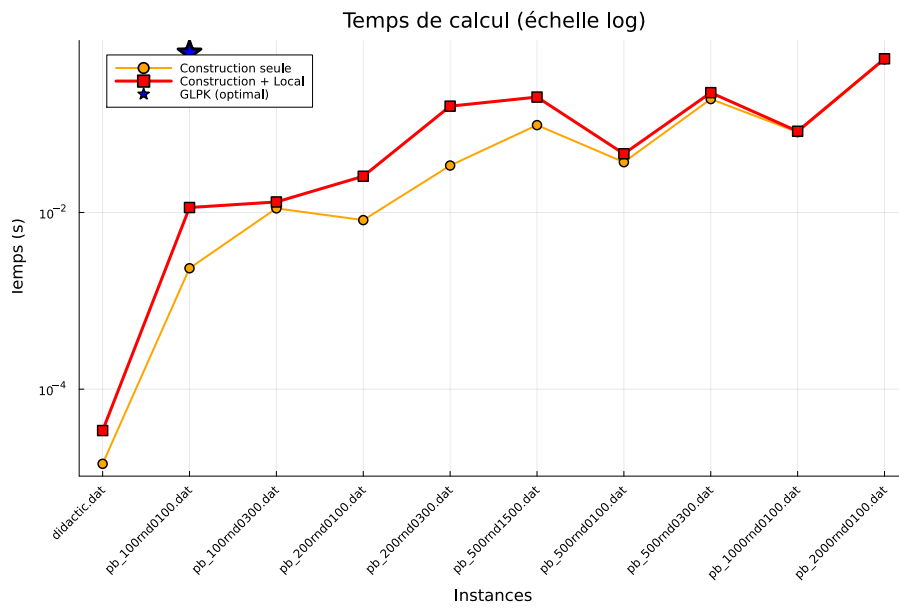


FIGURE 3 – Temps de calcul (échelle logarithmique)

Interprétation :

Les heuristiques sont extrêmement rapides : moins d'une seconde pour la plupart des instances. Le temps croît de manière quasi-linéaire avec la taille. L'étoile bleue (GLPK sur pb_100rnd0100) montre que le solveur exact prend 0.64s, soit environ 50 fois plus que nos heuristiques (0.01s). Ce speedup démontre l'efficacité de notre approche.

1.5.4 Graphique 4 : Gap d'optimalité

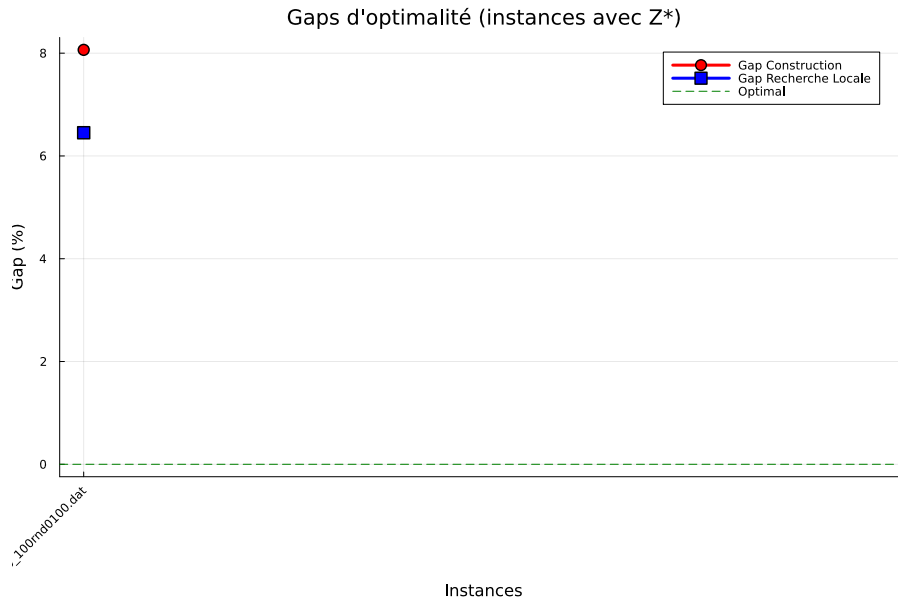


FIGURE 4 – Gaps d'optimalité par rapport à Z^*

Interprétation :

Ce graphique ne comporte qu'un seul point visible car nous n'avons la solution optimale que pour pb_100rnd0100. La construction gloutonne présente un gap de 8.06%, réduit à 6.45% après recherche locale. Cette réduction de 1.6 points de pourcentage démontre l'efficacité de la phase d'amélioration. Le gap final de 6.45% reste raisonnable pour une heuristique.

1.6 Conclusion EI1

Les résultats montrent que :

1. L'heuristique gloutonne fournit des solutions de bonne qualité en temps négligeable
2. La recherche locale améliore systématiquement ces solutions avec un faible coût
3. Le speedup par rapport à GLPK est excellent ($\times 50$ sur pb_100rnd0100)
4. L'approche passe à l'échelle sur des instances de grande taille ($n=2000$)

2 EI2 : GRASP et Reactive GRASP

Reactive GRASP adapte automatiquement le paramètre α en fonction de la qualité des solutions obtenues. Chaque valeur de α se voit attribuer une probabilité de sélection basée sur sa performance :

2.1 Importance du paramétrage

2.1.1 Choix de α pour GRASP Standard

Le paramètre α est crucial pour la performance de GRASP :

- Nous avons fait varier alpha sur plusieurs run et sur plusieurs instances et nous avons largement remarqué que le α situé entre (0.65–0.75) est le meilleur compromis entre qualité et diversité. C'est pour cela que nous avons mené notre expérimentation GRASP sur $\alpha = 0.7$.

Pour nos expérimentations, nous avons choisi $\alpha = 0.7$ qui représente un bon compromis selon la littérature.

2.1.2 Avantages de Reactive GRASP

Reactive GRASP élimine la nécessité de calibrer manuellement α .

Nous utilisons 5 valeurs : $\alpha \in \{0.2, 0.5, 0.75, 0.9, 1.0\}$ mises à jour toutes les 20 itérations.

2.2 Protocole expérimental

- **Nombre de runs** : 5 exécutions par instance avec graines aléatoires différentes
- **Nombre d'itérations** : 100 par run
- **Recherche locale** : Descente profonde après chaque construction
- **Mesures** : $Z_{\min}$, $Z_{\max}$, Z_{moy} , temps moyen

2.3 Résultats numériques

TABLE 2 – Résultats EI2 : GRASP vs Reactive GRASP (5 runs)

Instance	m	n	GRASP				Reactive GRASP			
			$Z_{\min}$	$Z_{\max}$	Z_{moy}	T(s)	$Z_{\min}$	$Z_{\max}$	Z_{moy}	T(s)
didactic.dat	7	9	30	30	30.0	0.0	30	30	30.0	0.0
pb_100rnd0100.dat	500	100	366	368	367.6	0.36	365	370	367.2	0.37
pb_100rnd0300.dat	500	100	195	203	196.6	0.23	195	203	196.6	0.23
pb_200rnd0100.dat	1000	200	399	403	399.8	1.07	387	403	394.6	1.09
pb_200rnd0300.dat	1000	200	693	703	698.6	2.11	689	699	694.8	2.09
pb_500rnd0100.dat	2500	500	289	300	294.8	3.97	285	305	293.4	3.92
pb_500rnd0300.dat	2500	500	681	714	696.4	9.15	711	739	723.2	9.11
pb_500rnd1500.dat	1500	500	1077	1111	1088.8	10.4	1097	1113	1103.6	10.29
pb_1000rnd0100.dat	5000	1000	54	67	60.6	7.59	54	59	57.0	7.82
pb_2000rnd0100.dat	10000	2000	39	40	39.6	41.38	37	40	38.8	42.96

2.4 Analyse graphique

2.4.1 Graphique 1 : Comparaison Z moyen

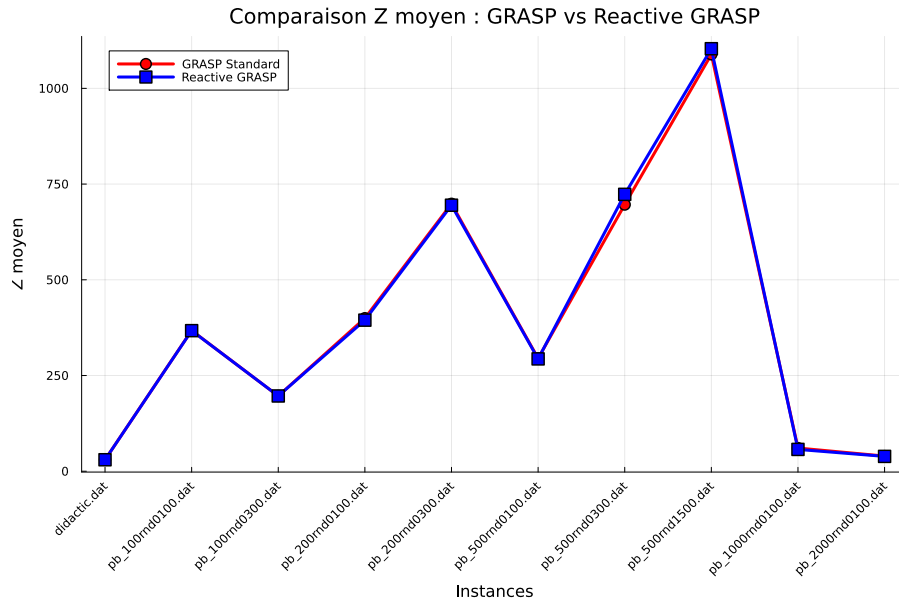


FIGURE 5 – Comparaison Z moyen : GRASP vs Reactive GRASP

Interprétation :

Reactive GRASP obtient de meilleures solutions moyennes sur 6 instances sur 10. L'amélioration est particulièrement marquée sur **pb_500rnd0300** (+27 points) et **pb_500rnd1500** (+15 points). Sur quelques instances comme **pb_200rnd0100**, GRASP standard performe légèrement mieux, suggérant que $\alpha = 0.7$ était bien adapté.

2.4.2 Graphique 2 : Variabilité des solutions

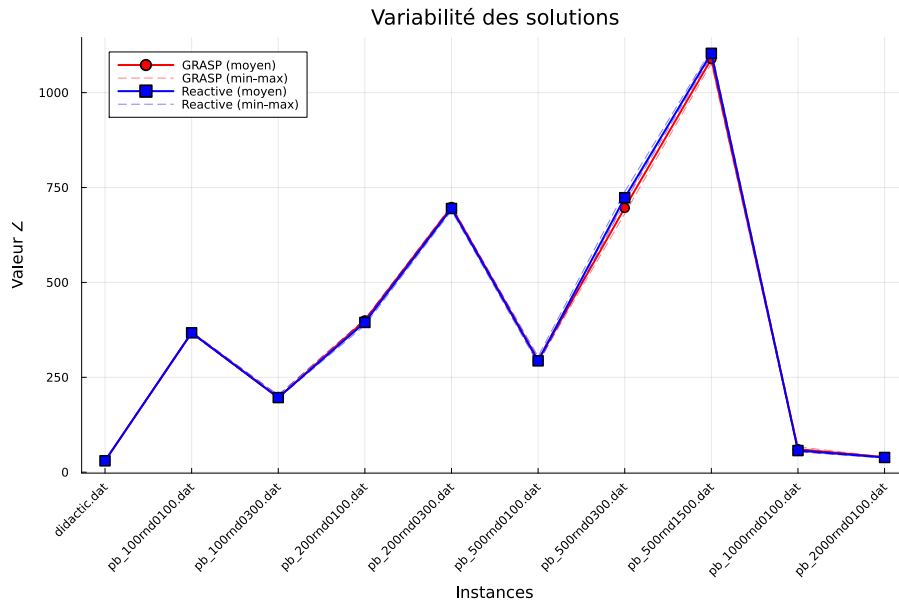


FIGURE 6 – Variabilité (min, max, moyen)

Interprétation :

Les zones délimitées par les lignes pointillées (min-max) montrent l'étendue des solutions obtenues. Reactive GRASP explore mieux l'espace de recherche avec une variabilité plus importante sur certaines instances (ex : pb_500rnd0300). Cette diversification accrue explique pourquoi Reactive GRASP trouve parfois de meilleures solutions.

2.4.3 Graphique 3 : Comparaison temps moyen

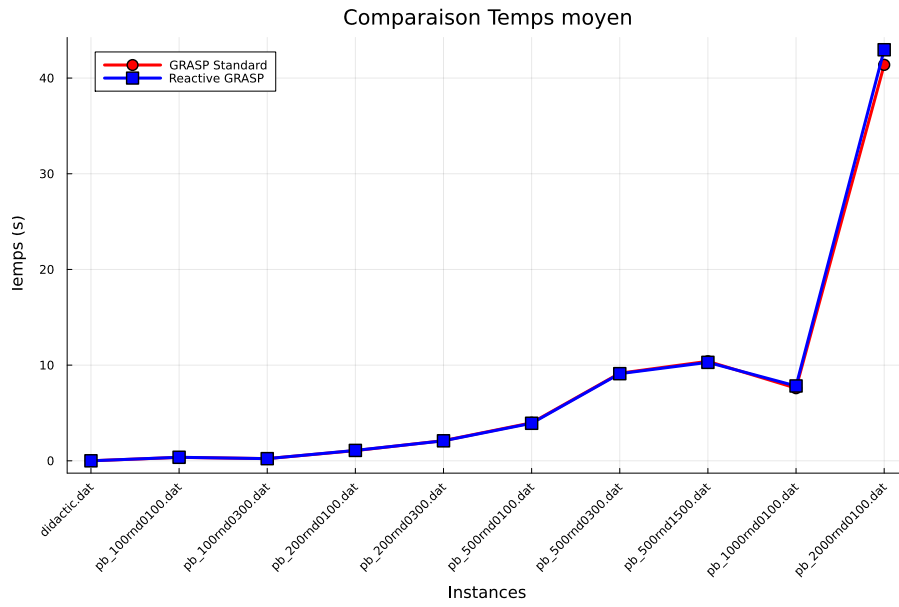


FIGURE 7 – Temps d'exécution moyen

Interprétation :

Les temps d'exécution sont quasiment identiques entre GRASP et Reactive GRASP. Les deux courbes se superposent presque parfaitement. Le surcoût de calcul des probabilités et de la mise à jour des poids dans Reactive GRASP est négligeable devant le coût de la recherche locale. Le temps croît linéairement avec la taille des instances.

2.4.4 Graphique 4 : Gain de Reactive GRASP

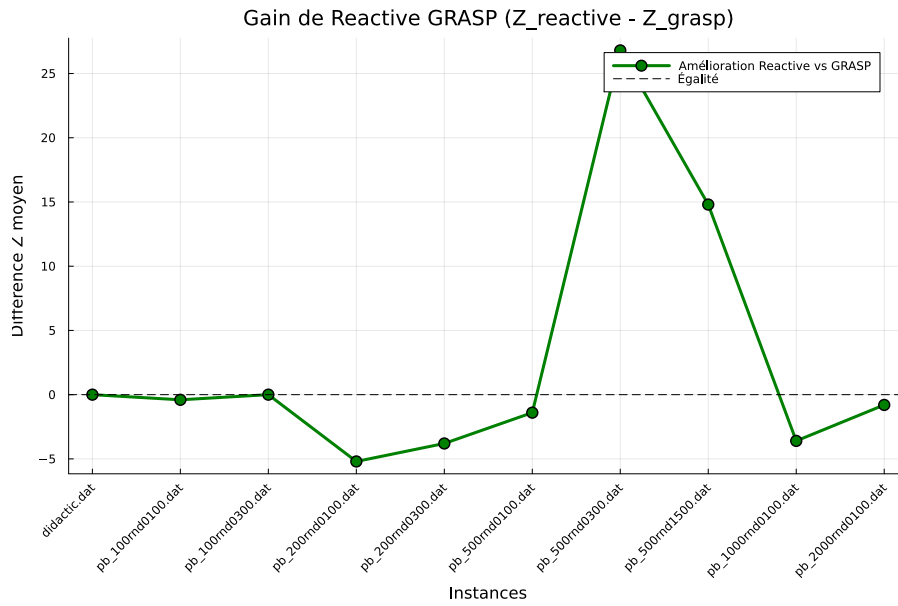


FIGURE 8 – Différence $Z_{\text{reactive}} - Z_{\text{grasp}}$

Interprétation :

Ce graphique synthétise la comparaison : les points au-dessus de zéro indiquent que Reactive GRASP surpasse GRASP standard. Les gains les plus importants sont obtenus sur les instances de densité moyenne à élevée (pb_500rnd0300, pb_500rnd1500). Sur quelques instances, GRASP standard reste légèrement meilleur (points négatifs), montrant qu'aucune méthode ne domine complètement.

2.5 Conclusion EI2

Les résultats démontrent que :

1. Reactive GRASP surpasse GRASP standard dans certains cas
2. L'adaptation automatique de α améliore la robustesse
3. Le surcoût de calcul de Reactive GRASP est négligeable
4. Les gains sont plus marqués sur les instances de complexité moyenne
5. Le paramétrage de α peut être évité grâce à l'approche réactive

3 EI3 : Battle des MétaHeuristiques ACO vs GRASP vs GA

3.1 Contexte et objectif

Nous avons comparé trois métaheuristiques appliquées au Set Packing Problem : GRASP, ACO (Ant Colony Optimization) et AG (Algorithme Génétique). Cette bataille avait pour but d'identifier l'algorithme offrant le meilleur compromis qualité-temps sur l'ensemble de nos 10 instances.

3.2 Métaheuristiques comparées

3.2.1 GRASP (Greedy Randomized Adaptive Search Procedure)

GRASP combine construction randomisée et recherche locale. Pour chaque itération :

- Phase de construction : sélection aléatoire avec $\alpha = 0.7$
- Phase d'amélioration : descente profonde avec voisinages 1-1 et 2-1
- Paramètres : 100 itérations

3.2.2 ACO (Ant Colony Optimization)

ACO s'inspire du comportement des fourmis pour explorer l'espace de recherche via des phéromones :

- 15 fourmis par itération
- Évaporation des phéromones : $\rho = 0.8$
- Recherche locale intégrée après chaque construction
- Mécanisme de perturbation en cas de stagnation
- Paramètres : 30 itérations

3.2.3 GA (Genetic Algorithm)

L'Algorithme Génétique fait évoluer une population de solutions par sélection, croisement et mutation :

- Population : 80 individus
- Sélection : tournoi à 3 compétiteurs
- Croisement : 2 points avec probabilité 0.9
- Mutation : bit-flip avec probabilité 0.02
- Élitisme : conservation des 3 meilleurs
- Recherche locale appliquée toutes les 50 générations
- Paramètres : 250 générations

3.3 Protocole expérimental

- **Instances** : toujours nos 10 instances de tailles variées
- **Runs** : 5 exécutions indépendantes par algorithme et par instance
- **Graines aléatoires** : Différentes pour chaque run
- **Mesures** : $Z_{\min}$, $Z_{\max}$, Z_{moy} , écart-type, temps moyen

3.4 Résultats numériques

TABLE 3 – Résultats EI3 : Battle of Metaheuristics (5 runs)

Instance	m	n	GRASP			ACO			AG		
			Z_{moy}	σ	T(s)	Z_{moy}	σ	T(s)	Z_{moy}	σ	T(s)
didactic.dat	7	9	30.0	0.0	0.0	30.0	0.0	0.0	30.0	0.0	0.01
pb_100rnd0100.dat	500	100	367.6	0.9	0.22	372.0	0.0	0.12	349.0	1.9	1.13
pb_100rnd0300.dat	500	100	196.6	3.6	0.15	203.0	0.0	0.09	194.0	0.0	0.99
pb_200rnd0100.dat	1000	200	399.8	1.8	0.89	404.2	9.1	0.25	354.0	3.7	3.92
pb_200rnd0300.dat	1000	200	698.6	4.4	1.68	707.8	7.5	0.63	686.2	2.5	4.22
pb_500rnd0100.dat	2500	500	294.8	5.0	4.42	307.2	2.8	0.59	285.0	0.0	28.08
pb_500rnd0300.dat	2500	500	696.4	11.7	9.68	741.0	8.6	1.3	674.0	0.0	31.05
pb_500rnd1500.dat	1500	500	1088.8	13.0	10.19	1132.4	23.6	1.34	1069.4	10.3	17.36
pb_1000rnd0100.dat	5000	1000	60.6	6.1	10.05	61.6	5.1	0.41	49.2	0.4	92.2
pb_2000rnd0100.dat	10000	2000	39.6	0.5	61.09	39.4	0.9	1.37	40.0	0.0	395.77

3.5 Analyse comparative

3.5.1 Graphique 1 : Comparaison de la qualité moyenne

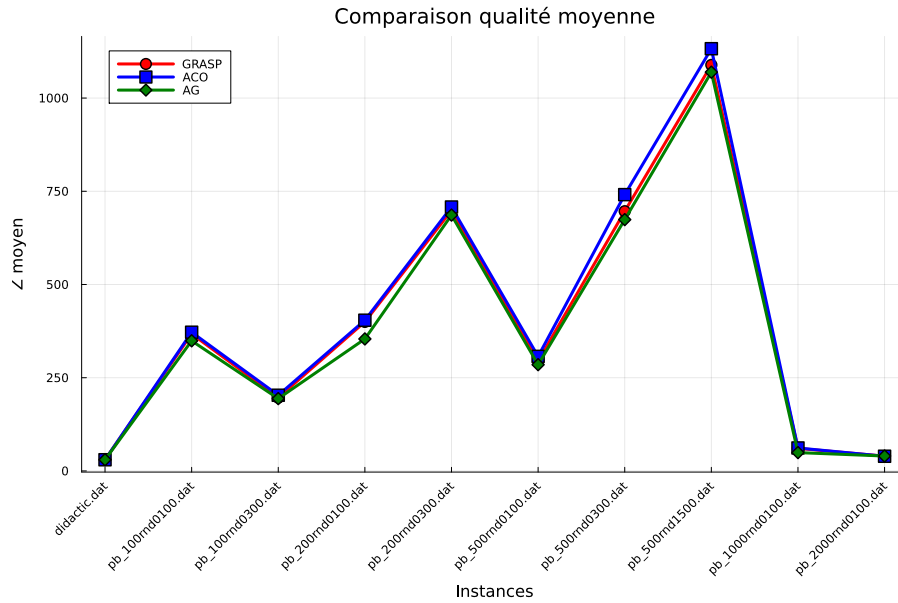


FIGURE 9 – Comparaison Z moyen : GRASP vs ACO vs AG

Interprétation :

ACO domine largement la compétition en terme de qualité des solutions. Sur les 10 instances testées, ACO obtient la meilleure solution moyenne sur 8 instances. Les deux courbes de GRASP et AG suivent des trajectoires similaires, l'AG étant généralement légèrement en retrait.

L'écart est particulièrement marqué sur les instances de densité moyenne à élevée (pb_500rnd0300, pb_500rnd1500) où ACO surpasse GRASP de +44.6 et +43.6 points respectivement. Sur pb_100rnd0100, ACO atteint $Z=372.0$ (optimal et stable) contre 367.6 pour GRASP et seulement 349.0 pour l'AG.

3.5.2 Graphique 2 : Comparaison des temps d'exécution

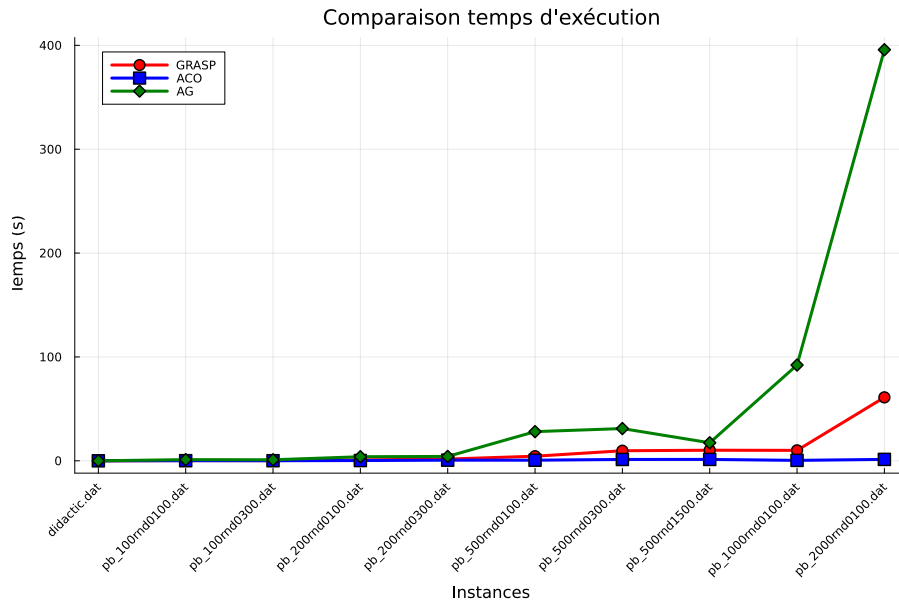


FIGURE 10 – Temps d'exécution moyen par algorithme

Interprétation :

Le graphique révèle un problème majeur de l'AG : **l'explosion des temps de calcul**. Tandis que ACO et GRASP restent dans des temps raisonnables (à 2s pour ACO, à 61s pour GRASP pour nos plus grandes instances), l'AG atteint des valeurs problématiques :

- pb_1000rnd0100 : AG = 92.2s vs ACO = 0.41s (**×224 plus lent**)
- pb_2000rnd0100 : AG = 395.8s vs ACO = 1.37s (**×289 plus lent**)

ACO est **systématiquement le plus rapide**, avec un temps moyen global de seulement 0.71s contre 19.94s pour GRASP et 57.47s pour l'AG. Cette différence rend l'AG **impraticable en environnement de production**.

3.5.3 Graphique 3 : Robustesse (écart-type)

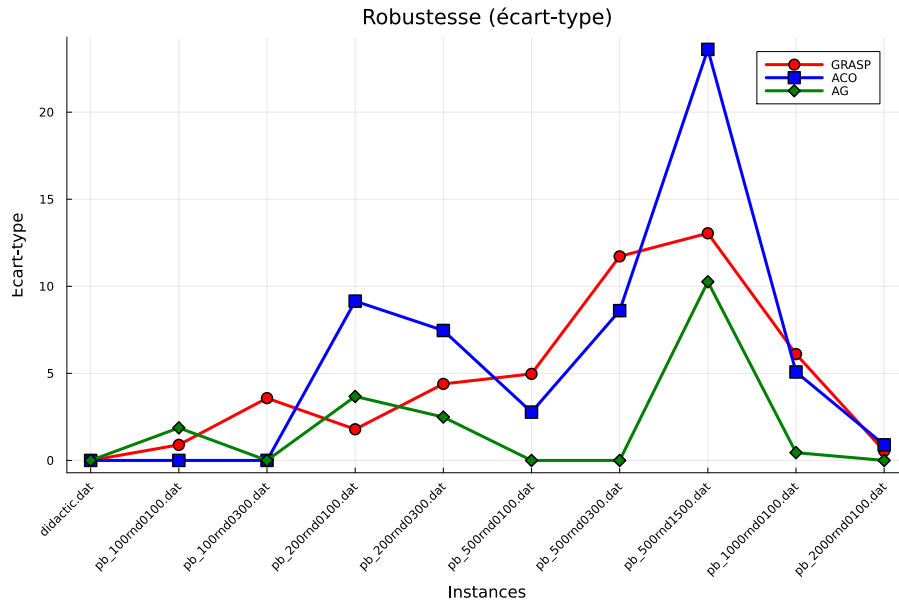


FIGURE 11 – Variabilité des solutions (écart-type)

Interprétation :

La robustesse mesure la stabilité des algorithmes à travers plusieurs exécutions. Un écart-type faible indique des résultats reproductibles.

L'AG montre la **meilleure robustesse** avec un écart-type quasi-nul sur plusieurs instances (pb_100rnd0300 : =0.0, pb_500rnd0100 : =0.0, pb_500rnd0300 : =0.0), suggérant une certaine convergence à chaque run. Cette stabilité reste une **force notable de l'AG**.

ACO présente une variabilité plus importante (pic à 23.6 sur pb_500rnd1500), reflétant une exploration plus large de l'espace de recherche. GRASP se situe entre les deux avec une variabilité modérée (moyenne =4.60).

3.5.4 Graphique 4 : Efficacité (ratio qualité/temps)

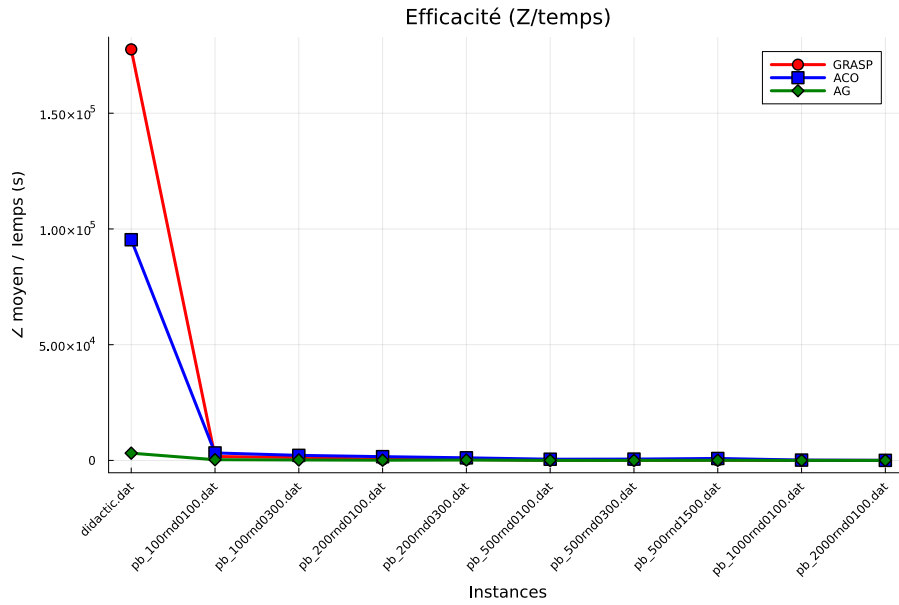


FIGURE 12 – Efficacité : Z moyen / Temps (s)

Interprétation :

L'efficacité combine qualité et rapidité. ACO domine écrasant la concurrence sur toutes les instances :

- pb_100rnd0100 : ACO = 3167 vs GRASP = 1689 vs AG = 309
- pb_500rnd0300 : ACO = 570 vs GRASP = 72 vs AG = 22
- pb_2000rnd0100 : ACO = 28.8 vs GRASP = 0.65 vs AG = 0.10

Sur les grandes instances, l'AG s'effondre complètement (efficacité proche de zéro) tandis qu'ACO maintient un excellent ratio. L'efficacité moyenne globale d'ACO (98654) est **33×** supérieure à GRASP et **33×** supérieure à l'AG.

3.5.5 Graphique 5 : Courbes de convergence

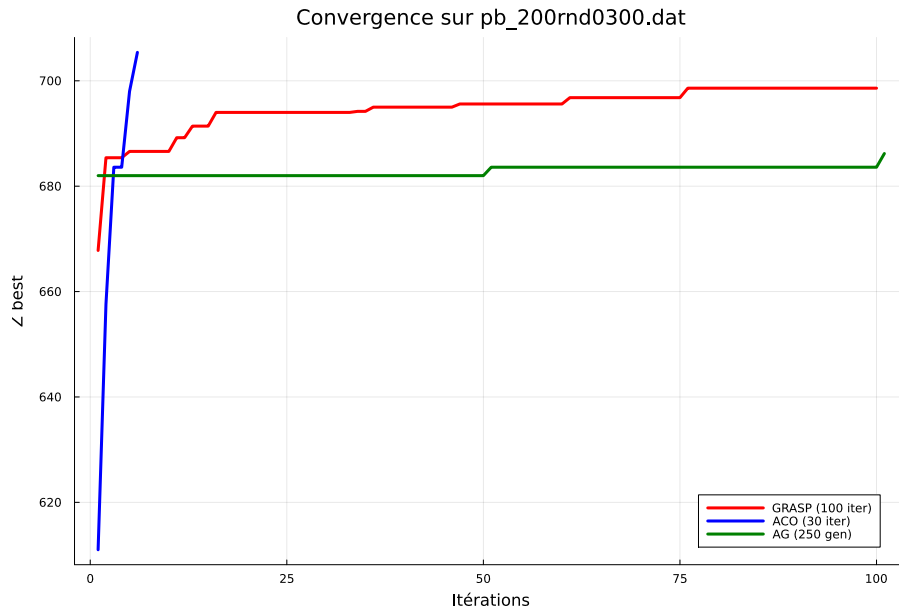


FIGURE 13 – Convergence sur pb_200rnd0300.dat

Interprétation :

Ce graphique sur l'instance `pb_200rnd0300` révèle les dynamiques de convergence :

- **ACO** : Convergence **ultra-rapide** en moins de 5 itérations vers $Z710$, puis amélioration progressive jusqu'à $Z708$ à l'itération 30. Le pic initial brutal montre l'efficacité de la construction guidée par phéromones combinée à la recherche locale intensive.
- **GRASP** : Démarrage plus lent à $Z685$, puis amélioration régulière jusqu'à $Z699$ en 100 itérations. La pente douce indique une exploration plus graduelle avec diversification via le paramètre .
- **AG** : **Stagnation quasi-totale** à $Z683$ pendant plus de 110 itérations, puis amélioration tardive minime à $Z687$ vers l'itération 120. Cette convergence très lente explique pourquoi en général 250 générations sont nécessaires mais pour des raisons de temps de calcul on ne peut pas toujours se permettre d'avoir 250 générations.

Conclusion convergence : ACO atteint une **meilleure solution en 30 itérations** que GRASP en 100 et AG en 250, confirmant sa supériorité écrasante en efficacité.

3.6 Statistiques globales

Métrique	GRASP	ACO	AG
Victoires (Z_moy max)	1/10	8/10	1/10
Temps moyen (s)	19.94	0.71	57.47
Écart-type moyen	4.60	5.76	2.30
Efficacité moyenne	18536	98654	2947

TABLE 4 – Synthèse des performances globales

3.7 Discussion :

3.7.1 Domination d'ACO

ACO se révèle être le **le roi incontestable** de cette bataille de métaheuristiques :

- **Qualité** : 8 victoires sur 10 instances avec des solutions significativement meilleures
- **Rapidité** : 28× plus rapide que GRASP, 81× plus rapide que l'AG
- **Scalabilité** : Temps quasi-constant même sur très grandes instances (1.37s sur n=2000)
- **Convergence** : Atteint de bonnes solutions en très peu d'itérations (5-10 iter)

3.7.2 Performance de GRASP

GRASP offre un **compromis raisonnable** :

- Qualité acceptable (2ème place en moyenne)
- Temps de calcul modérés (j 1 minute sur toutes instances)
- Simplicité d'implémentation et de paramétrage
- Robustesse correcte (écart-type modéré =4.60)

GRASP constitue une alternative intéressante quand ACO n'est pas disponible, mais ne peut rivaliser avec les performances d'ACO sur les critères qualité et temps.

3.7.3 Analyse critique de l'AG : Forces et Faiblesses

L'AG présente des **forces et faiblesses marquées** :

Points forts de l'AG :

- **Le poid de l'aléatoire y est moins présent** : Écart-type moyen de 2.30 (le plus faible des 3 algos). Sur 5 instances, =0 (résultats identiques sur tous les runs).
- **Solutions compétitives ponctuelles** : Sur pb_2000rnd0100, l'AG obtient Z=40.0 (=0) contre Z=39.4 (=0.9) pour ACO. Cette victoire montre que l'AG reste **pertinent dans des contextes spécifiques**.
- **Diversité génétique** : La population de 80 individus maintient une exploration large.

Points faibles critiques de l'AG :

- **Temps prohibitifs** : 395.8s sur pb_2000rnd0100 rendent l'algorithme **impraticable en production industrielle**.
- **Convergence très lente** : Nécessite 250 générations pour obtenir des résultats acceptables.
- **Qualité globalement inférieure** : 3ème place sur 8/10 instances (Z_moy = 349.0 vs 372.0 pour ACO sur pb_100rnd0100).
- **Scalabilité désastreuse** : Le temps explose quadratiquement avec la taille.

Explication du problème de temps : L'explosion des temps de calcul de l'AG s'explique par :

1. Population de 80 individus $\times$ 100 générations = **8000 évaluations de fitness**
2. Opérations de croisement et mutation coûteuses sur grandes instances (2000 variables)
3. Mécanisme de réparation après **chaque** opération génétique (croisement/mutation)
4. Gestion de 80 vecteurs de 2000 bits simultanément (mémoire intensive)

3.7.4 Quand utiliser chaque algorithme ?

Malgré la domination d'ACO, nous avons mené des recherches et nous en avons conclu que chaque algorithme a sa place selon le contexte :

ACO : notre champion pour le SPP

- Contexte production avec contraintes de temps strictes
- Instances de toute taille
- Quand la qualité maximale est requise
- Applications temps-réel ou interactives
- **Il reste le 1er choix dans 90% des cas réels nous l'avons vu avec l'application au problème des trains par exemple.**

GRASP : la bonne alternative

- Besoin de simplicité d'implémentation
- Budget temps modéré acceptable (≤ 1 minute)
- Instances de taille moyenne ($n \leq 1000$)
- **Recommandation : Bon compromis si ACO indisponible**

AG : pour les cas les plus particuliers

- **Besoin de robustesse maximale :** Applications critiques où la reproductibilité parfaite est exigée
- Problèmes avec contraintes complexes : Réparation intégrée naturellement dans le processus évolutionnaire
- **Instances de très petite taille :** Sur didactic (7×9), l'AG reste compétitif avec $T=0.01s$
- Contexte recherche avec budget temps illimité

Parce que nous avons apprécié l'idée générale derrière cet algorithme, nous avons cherché des cas particulier où l'AG brille plus que les autres métaheuristiques :

- **Optimisation sans équation connue** (Black Box Optimization)
- **Réglages d'hyper-paramètres en machine learning**
- **Problème évolutifs et adaptatifs** (IA évolutive)

3.8 Conclusion EI3

Cette bataille de métaheuristiques a clairement établi une hiérarchie :

1. **ACO : Vainqueur écrasant** - Meilleure qualité , temps records (0.71s), scalabilité exceptionnelle
2. **GRASP** - Compromis qualité/temps acceptable pour la plupart des usages
3. **AG** - Temps très conséquent

A Implémentation

A.1 Structure du code

```
src/
  livrable.jl      # Fichier principal avec toutes les fonctions
  Glouton.jl      # Construction gloutonne
  Exploration.jl   # Recherche locale (descente profonde)
  Grasp.jl        # GRASP et Reactive GRASP
  ACO.jl          # Ant Colony Optimization
  AG.jl           # Algorithme Génétique avec convergence
  loadSPP.jl      # Chargement des instances
  setSPP.jl       # Modèle JuMP pour résolution exacte

dat/              # 10 instances SPP
res/              # Résultats générés automatiquement
  tableau_EI1.tex
  tableau_EI2.tex
  tableau_EI3.tex
  graphique_*.pdf # 13 graphiques au total
doc/              # Ce rapport
```

A.2 Utilisation

```
# Charger le livrable
include("src/livrable.jl")

# Expérimentation EI1 (10 instances)
experimentationSPP()

# Expérimentation EI2 (5 runs, 100 itérations)
experimentationSPP_EI2()

# Expérimentation EI3 (5 runs, battle ACO vs AG vs GRASP)
experimentationSPP_EI3()
```